



Data Structures

Semester Project Report

Subject Name: Data Structures
Subject Code: COC-2074
Submitted To: Dr. Salman Raza / Sir Abdul Basit
Department: Department of Computer Science
Section: BSCS-B (3rd)
Date of Submission: 08-JAN-2026

Submitted By

NAME	REG. NO.
Muhammad Irtaza	24-NTU-CS-FL-1073
Shakeeb Ahmad	24-NTU-CS-FL-1093
Muhammad Adil	24-NTU-CS-FL-1054

Project Title:

Text File Compressor and Decompressor

Introduction:

With the rapid growth of digital data, efficient storage and transmission of text files has become increasingly important. File compression techniques help reduce file size by eliminating redundancy in data while preserving the original content.

This project implements **Huffman Coding**, a lossless data compression algorithm, using fundamental **data structures** concepts. The focus of the project is not only compression but also the correct and efficient use of **node-based data structures**, as required by the Data Structures course.

Problem Statement:

Text files often contain repeated characters, leading to inefficient storage when each character is stored using a fixed number of bits (e.g., ASCII encoding uses 8 bits per character regardless of frequency).

The problem is to design a system that:

- Compresses a text file by assigning shorter codes to frequently occurring characters
- Decompresses the file back to its original form without any data loss
- Uses node-based data structures only
- Efficiently handles large input files
- Uses file handling and produces verifiable output

Proposed Solution:

The proposed solution uses **Huffman Coding**, a greedy compression algorithm that assigns variable-length binary codes to characters based on their frequencies.

The solution works in the following phases:

1. Read the input text file
2. Count frequency of each character
3. Insert characters into a **node-based Min Heap**
4. Construct a Huffman Tree using the Min Heap
5. Generate binary codes for each character
6. Compress the file using generated codes
7. Store the Huffman Tree in the encoded file
8. Rebuild the tree during decoding
9. Decode the compressed file to restore the original text

This approach ensures **lossless compression**, meaning the decoded file is identical to the original file.

Core Data Structures Used:

Node-based Min Heap

Purpose:

- Acts as a priority queue to always extract the minimum frequency character
- Used during Huffman Tree construction

Why Min Heap?

- Huffman algorithm repeatedly requires the two smallest frequencies
- Min Heap provides efficient access to the minimum element

Why Node-Based?

- Dynamic memory allocation
- Direct integration of heap nodes into Huffman Tree

Huffman Tree (Binary Tree)

Purpose:

- Stores the hierarchical structure of character codes
- Used for encoding and decoding data

Characteristics:

- Leaf nodes represent characters
- Internal nodes represent merged frequencies
- Left edge = '0', Right edge = '1'

The Huffman Tree is constructed using the Min Heap and later traversed to generate binary codes.

Algorithm used for Compression and Decompression

Huffman Coding

Algorithm Steps for Huffman Tree construction:

1. Insert all characters with their frequencies into Min Heap
2. Extract two nodes with minimum frequency
3. Create a new node with combined frequency and the extracted nodes as its Internal Left and Right children
4. Insert the new node back into heap
5. Repeat until only one node remains

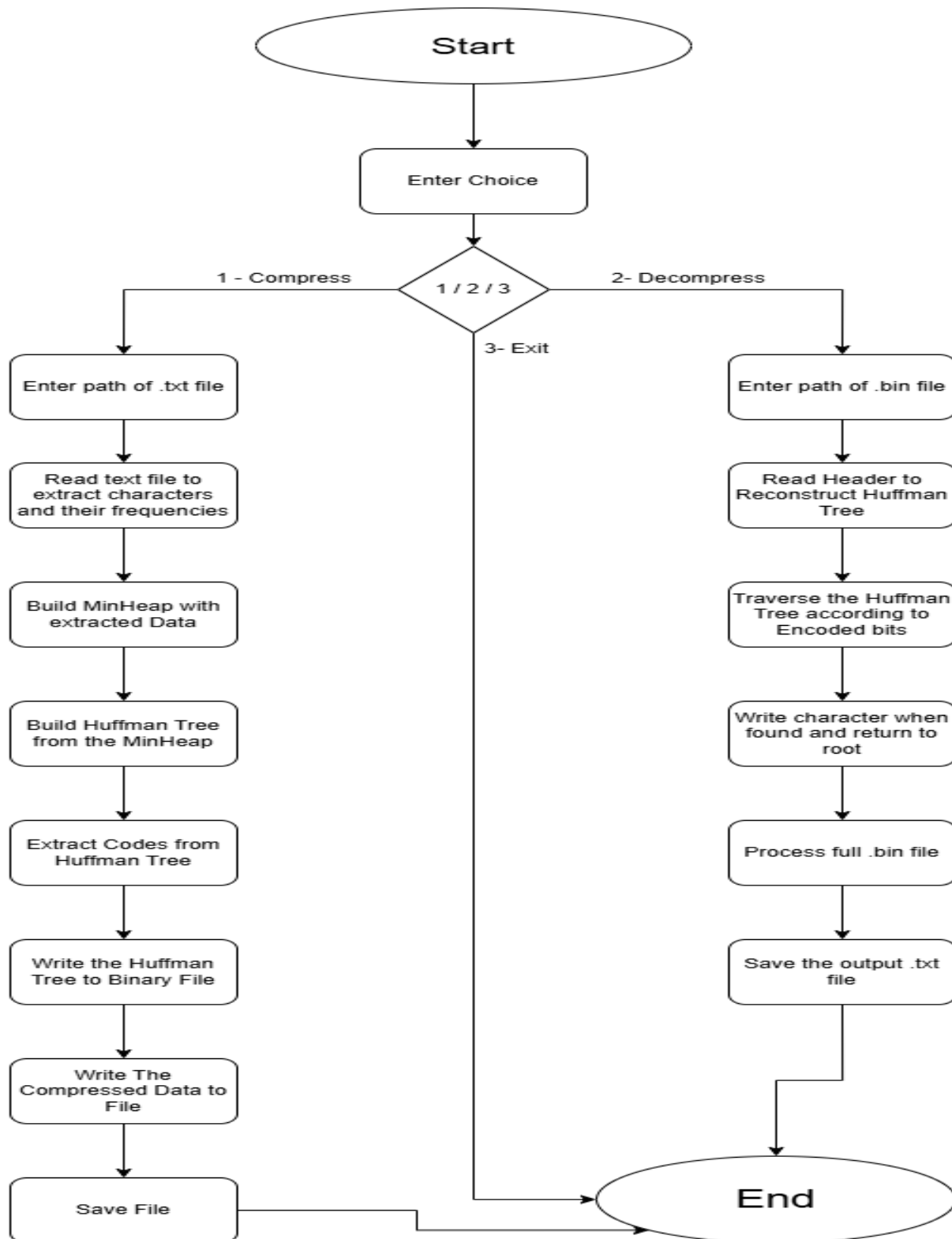
Encoding Algorithm

- Traverse Huffman Tree to generate binary codes
- Replace each character in input file with its corresponding code
- Write compressed binary data to output file

Decoding Algorithm

- Rebuild Huffman Tree from stored header
- Traverse tree using binary bits from compressed file
- Output original characters

Flowchart



Code

```
#include<iostream>

#include<fstream>

#include<bitset>

using namespace std;

template <typename T>

class ListNode {

    private:

        T data;

        ListNode<T>* next;

    public:

        ListNode(T d) { data = d;    next = NULL; }

        void setNext(ListNode<T>* n) { next = n; }

        void setData(T d) { data = d; }

        ListNode<T>* getNext() { return next; }

        T getData() { return data; }

};

template <typename T>
```

```

class List {
private:
    ListNode<T>* head;

    int size;
public:
    List() { head = NULL; size = 0; }

    int getSize() { return size; }

    void insert(T d)
    {
        ListNode<T>* newNode = new ListNode<T>(d);
        if (!head)
            head = newNode;

        else
        {
            ListNode<T>* temp = head;
            while (temp->getNext())
                temp = temp->getNext();

            temp->setNext(newNode);
        }
        size++;
    }

    T getAt(int index)

```



```

    {
        ListNode<T>* temp = head;
        for (int i = 0; i < index; i++)
            temp = temp->getNext();
        return temp->getData();
    }

void setAt(int index, T val)
    {
        ListNode<T>* temp = head;
        for (int i = 0; i < index; i++)
            temp = temp->getNext();
        temp->setData(val);
    }

int indexOf(T d)
    {
        ListNode<T>* temp = head;
        for (int i = 0; i < size; i++)
            {
                if (temp->getData() == d)
                    return i;
                temp = temp->getNext();
            }
        return -1;
    }

```

```
}  
};
```

```
class ReadTextFile {  
    private:  
        List<char> chars;  
        List<int> freqs;  
        int totalUnique;  
  
    public:  
        ReadTextFile() { totalUnique = 0; }  
  
        void Read(string path)  
        {  
            ifstream file(path.c_str());  
            if(!file)  
            {  
                cout<<"\n\nPath is not correct!!";  
                return;  
            }  
            char ch;  
            while (file.get(ch))  
            {
```

```

        int idx = chars.indexOf(ch);
        if (idx == -1)
            {
                chars.insert(ch);
                freqs.insert(1);
                totalUnique++;
            }
            else
                freqs.setAt(idx, freqs.getAt(idx) + 1);
        }
        file.close();
    }
    void getData(int &a, List<char> &c, List<int> &f)
    {
        for(int i = 0; i < totalUnique; i++)
        {
            c.insert(chars.getAt(i));
            f.insert(freqs.getAt(i));
        }
        a = totalUnique;
    }
};

```

```

class HeapNode{
    private:
        HeapNode *parent;
        HeapNode *left;
        HeapNode *right;
        HeapNode *internalLeft;
        HeapNode *internalRight;
        int value;
        char data;

    public:
        HeapNode(int v, char d = '\0')
        {
            value = v;
            data = d;
            parent = left = right = internalLeft = internalRight = NULL;
        }

        HeapNode* getLeft()    { return left; }
        HeapNode* getRight() { return right; }
        HeapNode* getParent() {    return parent; }
        HeapNode* getInternalLeft() { return internalLeft; }
        HeapNode* getInternalRight() { return internalRight; }

```

```

    int getValue() { return value; }
    char getData() { return data; }

    void setLeft(HeapNode *n) { left = n; }
    void setRight(HeapNode *n) { right = n; }
    void setParent(HeapNode *n) { parent = n; }
    void setinternalLeft(HeapNode *n) { internalLeft = n; }
    void setInternalRight(HeapNode *n) { internalRight = n; }
    void setValue(int v) { value = v; }
    void setData(char d) { data = d; }
};

```

```

string toBinary(int d)
{
    if(d == 0) return "0";

    string s = "";
    while(d != 0)
    {
        s += (d % 2) + '0';
        d /= 2;
    }
    for(int i = 0; i < s.length() / 2; i++)

```

```

        swap(s[i], s[s.length() - 1 - i]);

    return s;
}

class Heap{
    private:
        HeapNode *Root;
        int size;

    void swapNodes(HeapNode *a, HeapNode *b)
    {
        HeapNode* tempL = a->getInternalLeft();
        HeapNode* tempR = a->getInternalRight();
        int tempV = a->getValue();
        char tempD = a->getData();

        a->setInternalLeft(b->getInternalLeft());
        a->setInternalRight(b->getInternalRight());
        a->setValue(b->getValue());
        a->setData(b->getData());

        b->setInternalLeft(tempL);

```

```

        b->setInternalRight(tempR);
        b->setValue(tempV);
        b->setData(tempD);
    }
void HeapifyUp(HeapNode *n)
{
    if(!n || !n->getParent())
        return;

    if(n->getValue() < n->getParent()->getValue())
    {
        swapNodes(n, n->getParent());
        HeapifyUp(n->getParent());
    }
}
void HeapifyDown(HeapNode *n)
{
    if(!n) return;

    HeapNode *smallest = n;
    if(n->getLeft() && n->getLeft()->getValue() < smallest->getValue())
        smallest = n->getLeft();

```

```
        if(n->getRight() && n->getRight()->getValue() < smallest-  
>getValue())
```

```
            smallest = n->getRight();
```

```
        if(smallest != n)
```

```
        {
```

```
            swapNodes(n, smallest);
```

```
            HeapifyDown(smallest);
```

```
        }
```

```
    }
```

```
void display(HeapNode *n)
```

```
{
```

```
    if(!n) return;
```

```
    cout<<n->getData()<<"("<<n->getValue()<<")"<<" ";
```

```
    display(n->getLeft());
```

```
    display(n->getRight());
```

```
}
```

```
public:
```

```
    Heap() { Root = NULL; size = 0; }
```

```
    void Insert(HeapNode *n)
```



```

{
    size++;
    if(!Root)
    {
        Root = n;
        return;
    }

    HeapNode *parent = Root;
    string path = toBinary(size);
    for(int i = 1; i < path.length() - 1; i++)
        (path[i] == '0') ? parent = parent->getLeft() : parent = parent-
>getRight();

    if(path[path.length() - 1] == '0')
        parent->setLeft(n);
    else
        parent->setRight(n);

    n->setParent(parent);
    HeapifyUp(n);
}

HeapNode* ExtractRoot()

```

```

    {
        if(!Root) return NULL;

        HeapNode *res = new HeapNode(Root-&gtgetValue(), Root-
>getData());

        res->setInternalLeft(Root-&gtgetInternalLeft());
        res->setInternalRight(Root-&gtgetInternalRight());

        if(size == 1)
        {
            delete Root;
            Root = NULL;
            size--;
            return res;
        }

        HeapNode *curr = Root, *parent = NULL;
        string path = toBinary(size);
        for(int i = 1; i < path.length(); i++)
        {
            parent = curr;
            if(path[i] == '0')
                curr = curr-&gtgetLeft();

```

```

        else
            curr = curr->getRight();
    }
    swapNodes(Root, curr);

    if(parent->getLeft() == curr)
        parent->setLeft(NULL);
    else
        parent->setRight(NULL);

    delete curr;
    size--;
    HeapifyDown(Root);

    return res;
}

HeapNode *getRoot() { return Root; }
int getSize() { return size; }
void Display()
{
    cout<<"\nPreOrder : ";
    display(Root);
    cout<<"\n";
}

```

```

        }

};

class HuffmanTree{
    private:
        Heap *h;

        void GenerateCodes(HeapNode *n, string code, char *chars, string
*codes, int &idx)
        {
            if(!n) return;

            if(n->getData() != '\0')
            {
                chars[idx] = n->getData();
                codes[idx] = code;
                idx++;
                return;
            }

            GenerateCodes(n->getInternalLeft(), code + "0", chars, codes,
idx);

            GenerateCodes(n->getInternalRight(), code + "1", chars, codes,
idx);

        }

        void printPreorder(HeapNode* node)

```

```

    {
    if (node == NULL) return;

    if(node->getData() == '\0')
        cout<<"NULL"<<"("<<node->getValue()<<" )";
    else
        cout<<node->getData()<<"("<<node->getValue()<<" )";

    printPreorder(node->getInternalLeft());
    printPreorder(node->getInternalRight());
    }

```

public:

```

    HuffmanTree(Heap *H) { h = H; }
    void BuildHuffmanTree()
    {
        while(h->getSize() > 1)
        {
            HeapNode *l = h->ExtractRoot();
            HeapNode *r = h->ExtractRoot();
            HeapNode *n = new HeapNode(l->getValue() + r-
>getValue(), '\0');
            n->setinternalLeft(l);

```

```

        n->setInternalRight(r);
        h->Insert(n);
    }
    Display();
}

void getCodes(char *chars, string *codes)
{
    int idx = 0;
    GenerateCodes(h->getRoot(), "", chars, codes, idx);
}

void serializeTree(HeapNode* n, ofstream& out)
{
    if (!n) return;

    if (n->getData() != '\0') {
        out.put('1');        // leaf
        out.put(n->getData()); // character
    } else {
        out.put('0');        // internal node
        serializeTree(n->getInternalLeft(), out);
        serializeTree(n->getInternalRight(), out);
    }
}
}

```

```

        HeapNode* getRoot() { return h->getRoot(); }

        void Display()
        {
            cout<<"\nPreorder:\n";
            printPreorder(h->getRoot());
            cout<<"\n\n";
        }
};

```

```

class WriteTextFile {
private:
    char* chars;
    string* codes;
    int noOfChar;

    string getCode(char c)
    {
        for (int i = 0; i < noOfChar; i++)
        {
            if (chars[i] == c)
                return codes[i];
        }
        return "";
    }
};

```

```
}
```

```
unsigned char BitStringToByte(const string& bits)
```

```
{
```

```
    bitset<8> b(bits);
```

```
    return (unsigned char)b.to_ulong();
```

```
}
```

```
public:
```

```
    WriteTextFile(char* ch, string* cd, int n)
```

```
{ chars = ch; codes = cd; noOfChar = n; }
```

```
void writeFile(string inputPath, string outputPath, HuffmanTree* tree)
```

```
{
```

```
    ifstream in(inputPath.c_str());
```

```
    ofstream out(outputPath.c_str(), ios::binary);
```

```
    if (!in || !out)
```

```
    {
```

```
        cout << "File error!\n";
```

```
        return;
```

```
    }
```



```
tree->serializeTree(tree->getRoot(), out);
```

```
out.put('#');
```

```
long long textLength = 0;
```

```
char temp;
```

```
while (in.get(temp)) textLength++;
```

```
in.clear();
```

```
in.seekg(0);
```

```
out.write(reinterpret_cast<char*>(&textLength), sizeof(textLength));
```

```
string buffer = "";
```

```
char curr;
```

```
while (in.get(curr))
```

```
{
```

```
    buffer += getCode(curr);
```

```
    while (buffer.length() >= 8)
```

```
{
```

```
        out.put(BitStringToByte(buffer.substr(0, 8)));
```

```

        buffer.erase(0, 8);
    }
}

if (!buffer.empty())
{
    while (buffer.length() < 8)
        buffer += "0";

    out.put(BitStringToByte(buffer));
}

in.close();
out.close();
}
};

```

```

class DecodeBinaryFile{
    private:
        string inputPath;
        string outputPath;

        HeapNode* deserializeTree(ifstream& in)

```

```

{
    char flag;
    in.get(flag);

    if (flag == '1')
    {
        char c;
        in.get(c);
        return new HeapNode(0, c);
    }

    HeapNode* n = new HeapNode(0);
    n->setInternalLeft(deserializeTree(in));
    n->setInternalRight(deserializeTree(in));
    return n;
}

void display(HeapNode *n)
{
    if(!n) return;

    if(n->getData() != '\0')
        cout<<"("<<n->getData()<<") ";
    else

```

```

        cout<<"(NULL) ";

        display(n->getInternalLeft());
        display(n->getInternalRight());
    }

public:
    DecodeBinaryFile(string in, string out)
    {
        inputPath = in;
        outputPath = out;
    }

    void Decode()
    {
        ifstream in(inputPath.c_str(), ios::binary);
        ofstream out(outputPath.c_str());

        HeapNode* root = deserializeTree(in);

        cout<<"Step 1: Reading data from file and reconstructing
Huffman Tree:\n";

        system("pause");

        cout<<"PreOrder: ";

```

```

display(root);

cout<<"\n\n";

char sep;

in.get(sep);


long long textLength;

in.read(reinterpret_cast<char*>(&textLength), sizeof(textLength));


cout<<"Step 2: Traversing the Huffman Tree with respect to
compressed data bits from binary files:\n";

system("pause");


HeapNode* curr = root;

long long count = 0;

char byte;


while(in.get(byte) && count < textLength)
{
    for(int i = 7; i >= 0; i--)
    {
        bool bit = (byte >> i) & 1;

        curr = bit ? curr->getInternalRight() : curr->getInternalLeft();
    }
}

```

```

        if(curr->getData() != '\0')
        {
            out.put(curr->getData());
            count++;
            cout<<curr->getData();
            curr = root;
            if(count == textLength) break;
        }
    }
}

cout<<"\nStep 3:Writing this Data to "<<outputPath<<endl;
system("pause");
cout<<"\nData Written!!";
in.close(); out.close();

}

};

```

```

void Compress()
{
    cout<<"Enter the path of the Text File: \n";
    cout<<">> ";
    string txt;
    cin>>txt;
}

```

```
cout<<"\nStep 1: Reading Text File\n";
system("pause");

ReadTextFile reader;
reader.Read(txt);

int noOfChar;
List<char> chars;
List<int> freqs;

reader.getData(noOfChar, chars, freqs);

if(noOfChar == 0)
    { cout<<"File Empty!!"; return; }

cout<<"No of unique characters: "<<noOfChar;
cout<<"\n\nCharacter \tFrequency\n\n";
for(int i = 0; i < noOfChar; i++)
    cout<<" " <<chars.getAt(i)<<"\t\t " <<freqs.getAt(i)<<"\n";

cout<<"\n";
```

```
cout<<"Step 2: Building a MinHeap from obtained data:\n";
```

```
system("pause");
```

```
Heap *heap = new Heap();
```

```
for(int i = 0; i < noOfChar; i++)
```

```
{
```

```
    heap->Insert(new HeapNode(freqs.getAt(i), chars.getAt(i)));
```

```
    cout<<"\n\nAfter Inserting "<<chars.getAt(i)<<"("<<freqs.getAt(i)<<"\n";
```

```
    heap->Display();
```

```
}
```

```
cout<<"\nStep 3: Building Huffman Tree from the MinHeap:\n\n";
```

```
system("pause");
```

```
HuffmanTree *ht = new HuffmanTree(heap);
```

```
ht->BuildHuffmanTree();
```

```
char* finalChars = new char[noOfChar];
```

```
string* finalCodes = new string[noOfChar];
```

```
ht->getCodes(finalChars, finalCodes);
```

```
cout<<"\nStep 4: Extracting Codes:\n\n";
```

```
system("pause");
```



```

cout<<"Character\tCode\n\n";
for(int i = 0; i < noOfChar; i++)
    cout<<"  "<<finalChars[i]<<"\t\t"<<finalCodes[i]<<endl;

cout<<"\nStep 5: Write Compressed Binary File\n";
    system("pause");

WriteTextFile writer(finalChars, finalCodes, noOfChar);
string bin = txt;
    bin.erase(bin.length() - 4, 4);
    bin += "_compressed.bin";
writer.writeFile(txt, bin, ht);
cout<<"\nFile Written at: "<<bin;
}

void Decompress()
{
    cout<<"Enter the Path of the Binary File: ";
    cout<<">> ";
    string bin;
    cin>>bin;

    string txt = bin;
    txt.erase(txt.length() - 4, 4);

```

```

        txt += "_decompressed.txt";

        DecodeBinaryFile decoder(bin, txt);

        decoder.Decode();

    }

int main()
{
    cout<<"Text File Compressor and Decompressor\n\n";

    cout<<"\n\nEnter Your choice ( 1 / 2 / 3 ) : \n";
    cout<<"1 - Compress a Text File to Binary File\n";
    cout<<"2 - Decompress a Binary File to a Text File\n";
    cout<<"3 - Exit\n\n";

    int c;

    cout<<">> ";

    cin>>c;

    cout<<"\n\n";

    system("pause");

    switch(c)
    {
        case 1:
            Compress();

            break;
    }
}

```

```
        case 2:
            Decompress();
            break;
        case 3:
            return 0;
        default:
            cout<<"Invalid Choice!! Try Again\n\n";
            break;
    }
}
```

Simulation:

```
C:\Users\adila\OneDrive\Desktop >
Text File Compressor and Decompressor

Enter Your choice ( 1 / 2 / 3 ) :
1 - Compress a Text File to Binary File
2 - Decompress a Binary File to a Text File
3 - Exit

>> |
```

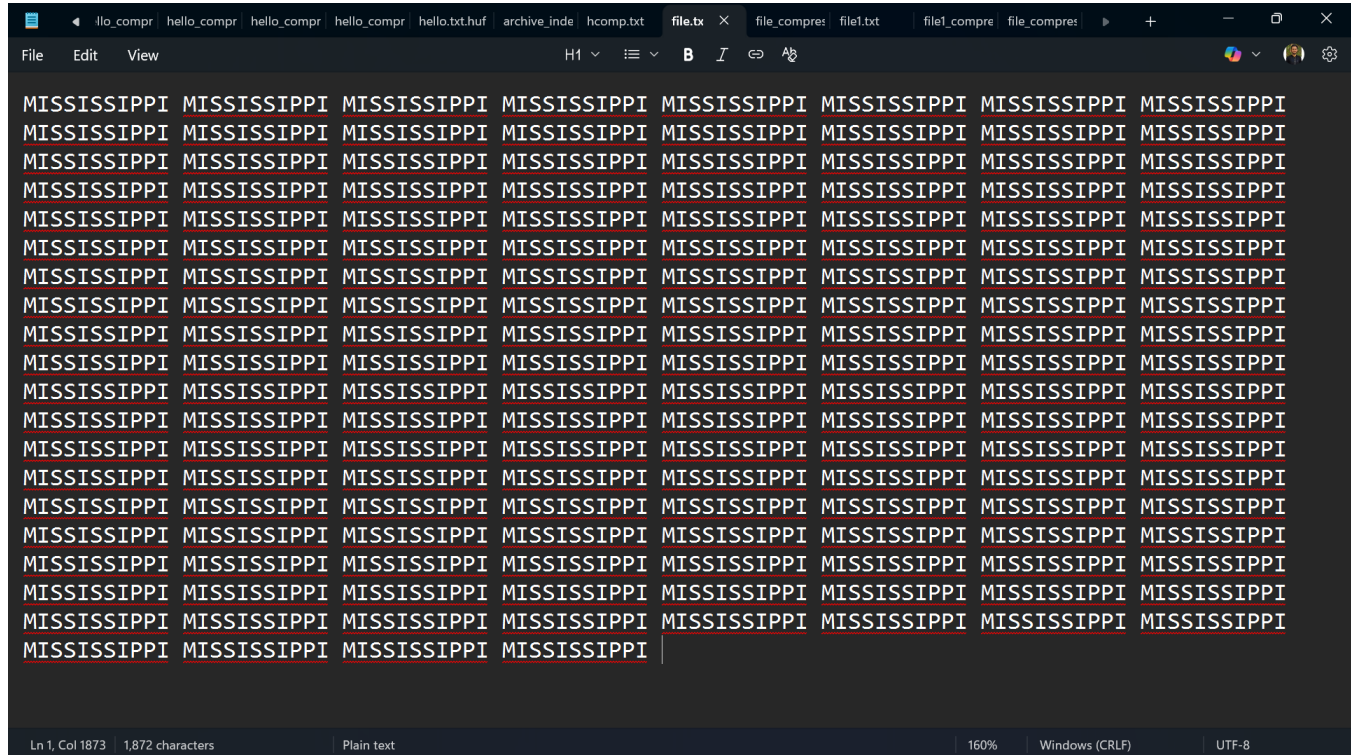
For Encoding:

```
C:\Users\adila\OneDrive\Desktop >
Enter Your choice ( 1 / 2 / 3 ) :
1 - Compress a Text File to Binary File
2 - Decompress a Binary File to a Text File
3 - Exit

>> 1

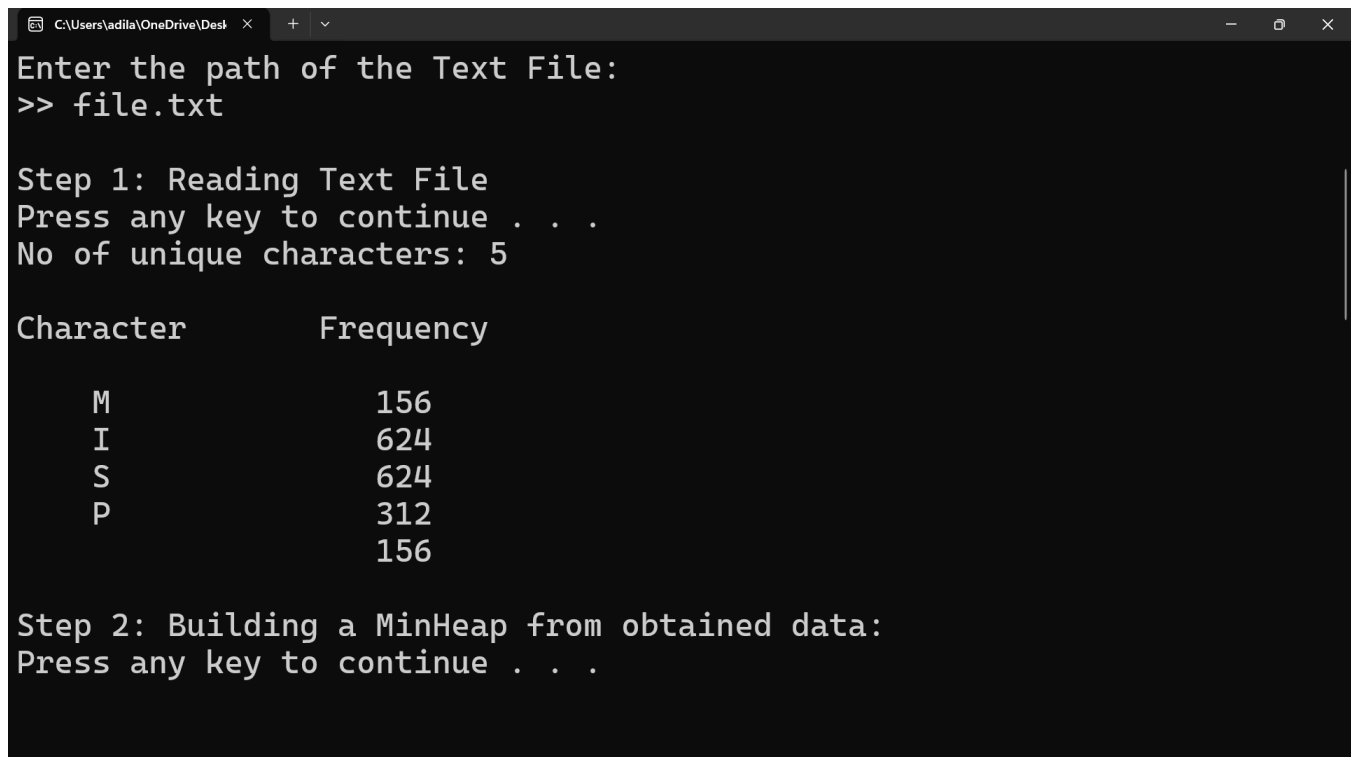
Press any key to continue . . .
Enter the path of the Text File:
>> file.txt
```

File.txt:



A screenshot of a text editor window with multiple tabs. The active tab is 'file.txt'. The text area contains the word 'MISSISSIPPI' repeated 1872 times in a single line. The status bar at the bottom indicates 'Ln 1, Col 1873 | 1,872 characters | Plain text | 160% | Windows (CRLF) | UTF-8'.

Console:



A screenshot of a console window showing the execution of a program. The output is as follows:

```
Enter the path of the Text File:
>> file.txt

Step 1: Reading Text File
Press any key to continue . . .
No of unique characters: 5

Character      Frequency
M              156
I              624
S              624
P              312
               156

Step 2: Building a MinHeap from obtained data:
Press any key to continue . . .
```

```
C:\Users\adila\OneDrive\Desk x + v - □ x

Step 2: Building a MinHeap from obtained data:
Press any key to continue . . .

After Inserting M(156)
PreOrder : M(156)

After Inserting I(624)
PreOrder : M(156) I(624)

After Inserting S(624)
PreOrder : M(156) I(624) S(624)

After Inserting P(312)
PreOrder : M(156) P(312) I(624) S(624)

After Inserting (156)
PreOrder : M(156) (156) I(624) P(312) S(624)
Step 3: Building Huffman Tree from the MinHeap:

C:\Users\adila\OneDrive\Desk x + v - □ x

PreOrder : M(156) (156) I(624) P(312) S(624)
Step 3: Building Huffman Tree from the MinHeap:
Press any key to continue . . .

Preorder:
NULL(1872) I(624) NULL(1248) S(624) NULL(624) P(312) NULL(312) M(156) (156)

Step 4: Extracting Codes:
Press any key to continue . . .
Character      Code
I              0
S              10
P              110
M              1110
               1111

Step 5: Write Compressed Binary File
Press any key to continue . . .

File Written at: file_compressed.bin
-----
```

Compressed File:

[illegible]

For Decompression:

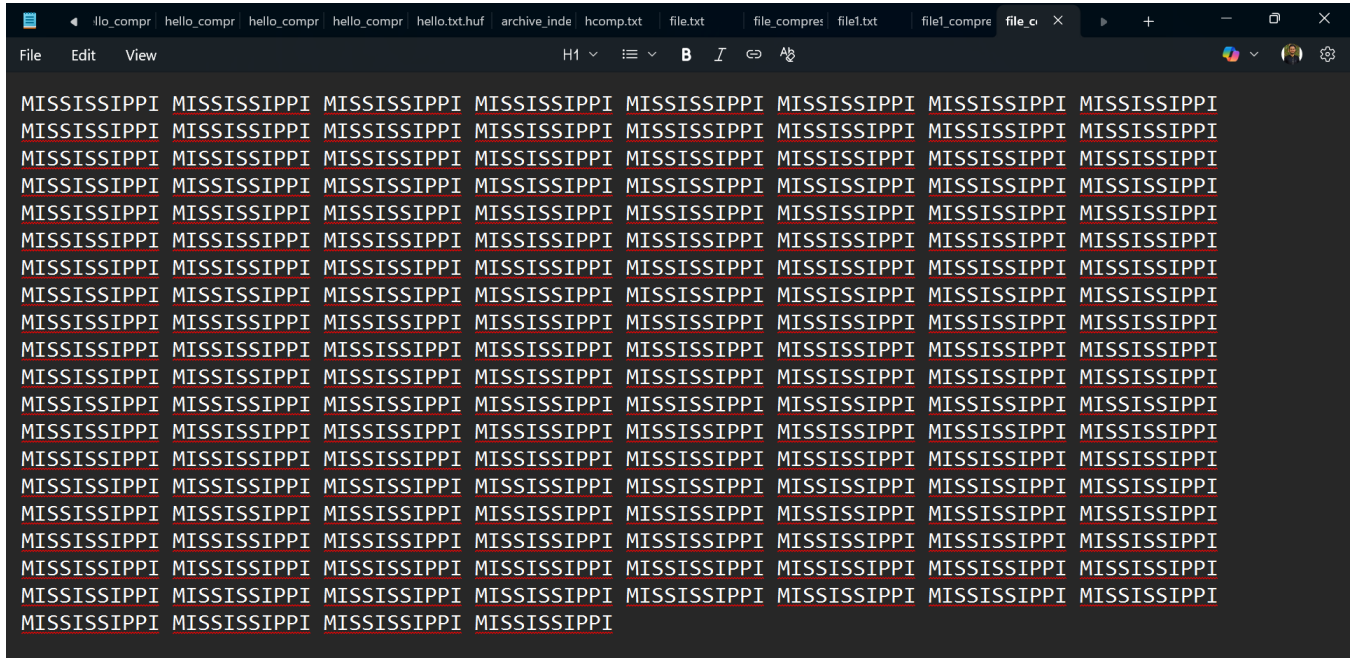
File_compressed.bin:

[illegible]

Console:

[illegible]

Decompressed file:



Result:

